Introduction to Linear Programming with Gurobipy

Solving Linear, Mixed-Integer, and Large-Scale Problems

INFORMS Student Chapter at Pitt

October 23, 2025

The Focus of Today

- After today you will know the basics of how to:
  - Solve the following in Python:
    - Linear Programs (LP).
    - Integer Programs (IP).
    - Mixed-Integer Linear Programs (MILP).
  - Handle large-scale problems that take days (or weeks!) to solve.

Why Gurobi?

- Gurobi is a state-of-the-art solver for mathematical programming.
- Handles LP, MILP, quadratic, and non-linear problems efficiently.
- Widely used in both academic research and industry applications.
- All features free for students and for academic research.
- Supports Python via the Gurobipy interface.

Installing Gurobi

1. Go to `https://www.gurobi.com`.
2. Use your Pitt email to register for a Gurobi account.
3. Go to `https://www.gurobi.com/downloads/gurobi-software/`
4. Download and install Gurobi based on your device.

Installing Gurobipy

1. Install the Gurobipy Python interface:
   - **For macOS/Linux (Terminal):**

     ```
     pip install gurobipy
     ```

     Make sure Gurobi is added to your PATH environment variables. You can do this by editing the system environment variables:

     ```
     export GUROBI_HOME="/Library/gurobi<version>"
     export PATH="${PATH}:${GUROBI_HOME}/bin"
     export LD_LIBRARY_PATH="${GUROBI_HOME}/lib"
     ```

   - **For Windows (Command Prompt or PowerShell):**

     ```
     pip install gurobipy
     ```

     Make sure to install Gurobi and add it to your PATH. You can do this by editing the system environment variables:

     ```
     setx GUROBI_HOME "C:\gurobi<version>"
     setx PATH "%PATH%;%GUROBI_HOME%\bin"
     setx LD_LIBRARY_PATH "%GUROBI_HOME%\lib"
     ```

Verifying the Installation

1. Verify the installation with a simple test:

    2. Open a Python interpreter or Jupyter notebook and run:

```
import gurobipy
```

- If no error occurs, the installation is successful.

Introduction to Linear Programming (LP)

- LP is the simplest form of mathematical programming.
- Objective: Minimize (or maximize) a linear function subject to linear constraints.
- Example: A diet problem, transportation problem.

LP Example: Diet Problem

A nutritionist aims to minimize the cost of a meal plan that satisfies having 500 calories and 30 grams of protein per day, with two food items, Food 1 and Food 2, where:

- Food 1 costs \$2/unit and provides 50 calories and 10 grams of protein per unit.
- Food 2 costs \$3/unit and provides 20 calories and 5 grams of protein per unit.

Let x_1 and x_2 be the quantities of Food 1 and Food 2, respectively. The optimization problem is:

$$\begin{aligned} \text{Minimize} \quad & 2x_1 + 3x_2 \\ \text{subject to} \quad & 50x_1 + 20x_2 \geq 500 \quad \text{(calories)} \\ & 10x_1 + 5x_2 \geq 30 \quad \text{(protein)} \\ & x_1, x_2 \geq 0 \end{aligned}$$

Gurobi Model and Variable Creation

- `model = gp.Model()`: Defines a Gurobi Model, named `model`.
- `model.addVar()`: Defines a single decision variable.
- `model.addVars()`: Defines a multiple decision variables.
- `vtype`: Specifies the variable type. Common options include:
  - `GRB.CONTINUOUS`: Continuous variable (default).
  - `GRB.BINARY`: Binary variable (0 or 1).
  - `GRB.INTEGER`: Integer variable.
- `lb & ub`: Set upper and lower bound on variable value:
  - `GRB.INFINITY`: Using infinity in Gurobi.

Example

```
model = gp.Model("name")
x = model.addVar(vtype=GRB.CONTINUOUS,
    lb=0, ub=GRB.INFINITY, name="x")
y = model.addVar(vtype=GRB.BINARY, name="y")
z = model.addVar(vtype=GRB.INTEGER, name="z")
```

Setting the Objective Function

- `model.setObjective()`: Sets the objective function.
- `GRB.MINIMIZE/GRB.MAXIMIZE`: Set the problem to be minimization/maximization.

Example

```
model.setObjective(2*x + 3, GRB.MINIMIZE)
```

Adding Constraints and Optimizing the Model

- `model.addConstr()`: Defines a single constraint.
- `model.addConstrs()`: Defines a multiple constraints.
- `model.optimize()`: Automatically picks one or a combination of these methods to optimize a problem:
  - Simplex
  - Branch and bound
  - Linear relaxation
  - etc.

Example

```
model = gp.Model("name")
x = model.addVar(name="x")
model.addConstr(x >= 5, "c1")
model.setObjective(2*x + 3, GRB.MINIMIZE)
model.optimize()
```

Accessing the Solution and Constraint Information

- `model.getVars()`: Stores the optimal solution.
- `[Your Variable].X`: The syntax to access the optimal solution to `[Your Variable]`.
- `[Your Variable].varName`: The syntax to access the name you've assigned to `[Your Variable]`.
- `model.getConstrByName("c1")`: Access the constraint by its name.

Example

```
for var in model.getVars():
    print(f'{var.varName}: {var.X}')
constraint = model.getConstrByName("c1")
print(f'Constraint Name: {constraint.ConstrName}')
```

Now, try to solve the Diet problem using Gurobi

LP Example: Answer

```
model = gp.Model("Diet")
# Add variables (for each food)
x1 = model.addVar(vtype=GRB.CONTINUOUS, name="Food1")
x2 = model.addVar(vtype=GRB.CONTINUOUS, name="Food2")

# Objective: minimize cost
model.setObjective(2*x1 + 3*x2, GRB.MINIMIZE)

# Constraints: nutrient requirements (Naming is optional)
model.addConstr(50*x1 + 20*x2 >= 500, "calories")
model.addConstr(10*x1 + 5*x2 >= 30, "protein")

model.optimize()

#Getting the optimal solutions
for v in model.getVars():
    print(f'{v.varName} {v.X}')
```

Mixed-Integer Linear Programming (MILP)

- MILP includes both continuous and integer decision variables.
- Example problems:
  - Knapsack problem: decide which items to include in a bag.
  - Scheduling problem: assign jobs to machines with constraints.

MILP Example: Knapsack Problem

- You are given a set of items, each with a weight and a profit. You need to determine which items to include in a knapsack so that the total weight does not exceed a given limit while maximizing the total profit.

$$\begin{aligned} \text{Maximize} \quad & 5x_1 + 8x_2 + 6x_3 \\ \text{subject to} \quad & 3x_1 + 6x_2 + 8x_3 \leq 10 \\ & x_1, x_2, x_3 \in \{0, 1\} \end{aligned}$$

- `vtype = GRB.BINARY`: Set a defined variable to be continuous.
- Use what you learned in the diet example to solve it using Gurobi!

MILP Example: Answer

```
model = gp.Model("Knapsack")
# Add variables (binary, either take or don't take an item)
x1 = model.addVar(vtype=GRB.BINARY, name="Item1")
x2 = model.addVar(vtype=GRB.BINARY, name="Item2")
x3 = model.addVar(vtype=GRB.BINARY, name="Item3")

# Objective: maximize profit
model.setObjective(5*x1 + 8*x2 + 6*x3, GRB.MAXIMIZE)

# Constraint: weight limit
model.addConstr(3*x1 + 6*x2 + 8*x3 <= 10, "capacity")

model.optimize()
```

Optimization Status and Objective Value

- After solving the model, check the optimization status using `model.Status`.
- The optimal solution is available only if `model.Status == GRB.OPTIMAL`.
- You can retrieve the optimal objective value with `model.ObjVal`.

Important Status Codes

- `GRB.OPTIMAL`: An optimal solution was found.
- `GRB.INFEASIBLE`: The model is infeasible.
- `GRB.UNBOUNDED`: The model is unbounded.
- `GRB.TIME_LIMIT`: The time limit was reached before finding an optimal solution.

Optimization Status and Objective Value: Example

- Take your code for the knapsack problem.

(A) After optimizing the model, add the following condition that:

  - Prints the optimal objective value in case the model solves to optimality
  - Prints the status of the model, otherwise.

(B) Change the right hand side to -1, and rerun your code. What do you observe?

Optimization Status and Objective Value: Answer

```
if model.Status == GRB.OPTIMAL:
    print(f"Optimal solution found with objective value:
        {model.ObjVal}")
else:
    print(f"Optimization ended with status: {model.Status}")
```

Defining Multiple Variables

- Use `model.addVars()` to define multiple decision variables (like x_{ij}) efficiently.
- Let $i \in \{1, \ldots, n\}$ and $j \in \{1, \ldots, m\}$.
- You can create variables as:

1. **List:**

```
x = model.addVars(n,m, vtype=GRB.BINARY, name="x")
```

2. **Dictionary:**

```
x = model.addVars(range(n),range(m), vtype=GRB.BINARY,
    name="x")
```

3. **NumPy Array:**

```
import numpy as np
x = model.addVars(np.arange(n) , np.arange(m),
    vtype=GRB.BINARY,name="x")
```

- Using dictionaries allows for better organization and easy access to variable values.

Defining Multiple Constraints of Multiple Variables

- You might have encountered a term in the objective function or a constraint:

$$\sum_{i=1}^{n} x_{ij} = 1 \quad \forall j \in \{1, \ldots, n\}$$

- Use `model.addConstrs()` to define multiple constraints in a concise manner.

```
model.addConstrs((gp.quicksum(x[i,j] for i in
    range(n)) == 1 for j in range(m)),
    name="constraint")
```

  - This method ensures each column j sums to 1, enforcing constraints efficiently.
  - Benefits of using `addConstrs()`:
    - Streamlined constraint addition.
    - Improved memory and processing efficiency for large models.

MILP Example: Medium-Scale Knapsack

- Revisit your code for the knapsack problem.
- Change it so you can solve a knapsack problem with 100 items
- Increase the capacity to 100
- You might want to generate random cost and weight for each item by setting:

```
seed = 0
n = 100
c = np.random.uniform(1,10,n)
a = np.random.uniform(1,10,n)
```

MILP Example: Medium-Scale Knapsack

```
np.random.seed(42)
n = 100
c = np.random.uniform(1,10,n)
a = np.random.uniform(1,10,n)
capacity = 100

model = gp.Model("Knapsack")
# Add variables (binary, either take or don't take an item)
x = model.addVars(range(n),vtype=GRB.BINARY, name="Items")

# Objective: maximize profit
model.setObjective(gp.quicksum(c[i]*x[i] for i in range(n))
    , GRB.MAXIMIZE)

# Constraint: weight limit
model.addConstr(gp.quicksum(a[i]*x[i] for i in range(n))
    <= capacity, "weight_limit")

model.optimize()
```

MILP Example: Large-Scale Knapsack

- Increase the capacity to 5000
- Increase n to 5000 and rerun the code

License Requirement

- Gurobi requires a license to solve larger problems.
- Small problems (like those with 100 variables) can be solved without a license.
- Larger instances, especially with mixed-integer constraints, need an academic license.

License Error Example

- When the problem size exceeds the free version limit, you get a license error:

```
GurobiError: Model too large for size-limited license.
```

Installing Gurobi License

1. Login to your Gurobi account.
2. Under `Download and Licenses` on the top bar, click on under `Your Gurobi Licenses`
3. Click on `Request` on the left bar.
4. Make sure you are connected to Pitt WiFi.
5. Select `Named-User Academic`.
6. You will get a license key like the following:

```
grbgetkey YOUR-LICENSE-KEY
```

7. Copy and paste the key to the Start/Run menu (Windows only) or a command/terminal prompt (any system).
8. Activate the license on your system.
9. To see whether it works rerun the code for $n = 5000$.

Large-Scale MILP Problem

- Real-world problems often involve large-scale MILP models.
- Example: supply chain optimization, production scheduling.
- Large-scale MILP challenges: memory, solving time, computational complexity.

Large-Scale Facility Location Problem

- Objective: Minimize the total cost of opening facilities and shipping products to customers.
- Decision variables:
  - y_i: Binary variable, 1 if facility i is opened, 0 otherwise.
  - x_{ij}: Continuous variable, representing amount shipped from facility i to customer j.
- Constraints:
  - Customers' demands must be met.
  - Shipping from a facility only if it's opened.

Facility Location Formulation

$$\text{Minimize} \quad \sum_i f_i y_i + \sum_i \sum_j c_{ij} x_{ij}$$

$$\text{Subject to} \quad \sum_i x_{ij} = d_j, \quad \forall j \quad \text{(Demand satisfaction)}$$

$$x_{ij} \leq M y_i, \quad \forall i, j \quad \text{(Big M constraint for shipping)}$$

$$x_{ij} \geq 0, \quad \forall i, j$$

$$y_i \in \{0, 1\}, \quad \forall i$$

Problem Setup and Random Parameters

- Randomly define problem parameters:
  - Number of facilities: 10.
  - Number of customers: 15.
  - c_{ij}: Shipping cost from facility i to customer j (uniformly distributed between 1 and 100).
  - f_i: Facility opening cost (uniformly distributed between 500 and 1000).
  - d_j: Customer j's demand (uniformly distributed between 50 and 100).
- Solve using Gurobi.

Facility location: Answer

```
number_of_facilities = 120
number_of_customers = 1000

c = np.random.uniform(1,100,(number_of_facilities,
    number_of_customers))

f = np.random.uniform(500,1e3,number_of_facilities)

d = np.random.uniform(50,100,number_of_customers)
```

Facility location: Answer

```
BigM = np.max(d)

model = gp.Model('FL_M')

x = model.addVars(range(number_of_facilities),
    range(number_of_customers),vtype=GRB.CONTINUOUS,
                    lb=0,ub=GRB.INFINITY,name='assgnmnt')

y = model.addVars(range(number_of_facilities),
    vtype=GRB.BINARY,name='opening')

model.setObjective(quicksum(f[i]*y[i] for i in
    range(number_of_facilities)) + quicksum(c[i,j]*x[i,j]
        for i in range(number_of_facilities)
            for j in range(number_of_customers)),GRB.MINIMIZE)
```

Facility location: Answer

```
model.addConstrs((gp.quicksum(x[i,j] for i in
    range(number_of_facilities)) == d[j]
        for j in range(number_of_customers)),
            name='demand_satisfaction')

model.addConstrs((x[i,j] <= BigM * y[i] for i in
    range(number_of_facilities) for j in
        range(number_of_customers)),
                name='BigM')

model.optimize()
```

Information in Gurobi Output

```
Gurobi Optimizer version 10.0.1 build v10.0.1rc0 (mac64[arm])
CPU model: Apple M2 Pro
Thread count: 12 physical cores, 12 logical processors, using up to 12 threads

Optimize a model with 151000 rows, 150150 columns and 450000 nonzeros
Model fingerprint: 0xa74d1e45
Variable types: 150000 continuous, 150 integer (150 binary)
Coefficient statistics:
  Matrix range     [1e+00, 1e+03]
  Objective range  [1e+00, 1e+03]
  Bounds range     [1e+00, 1e+00]
  RHS range        [5e+01, 1e+02]
Found heuristic solution: objective 3935770.2675
Presolve time: 0.25s
Presolved: 151000 rows, 150150 columns, 450000 nonzeros
Variable types: 150000 continuous, 150 integer (150 binary)
Deterministic concurrent LP optimizer: primal and dual simplex
Showing first log only...

Concurrent spin time: 0.00s

Solved with dual simplex

Root relaxation: objective 2.006108e+05, 4547 iterations, 2.51 seconds (17.00 work units)
```

Gurobi Version

Standard LP format

$$\min c^T x \quad Ax = b \quad x \geq 0$$

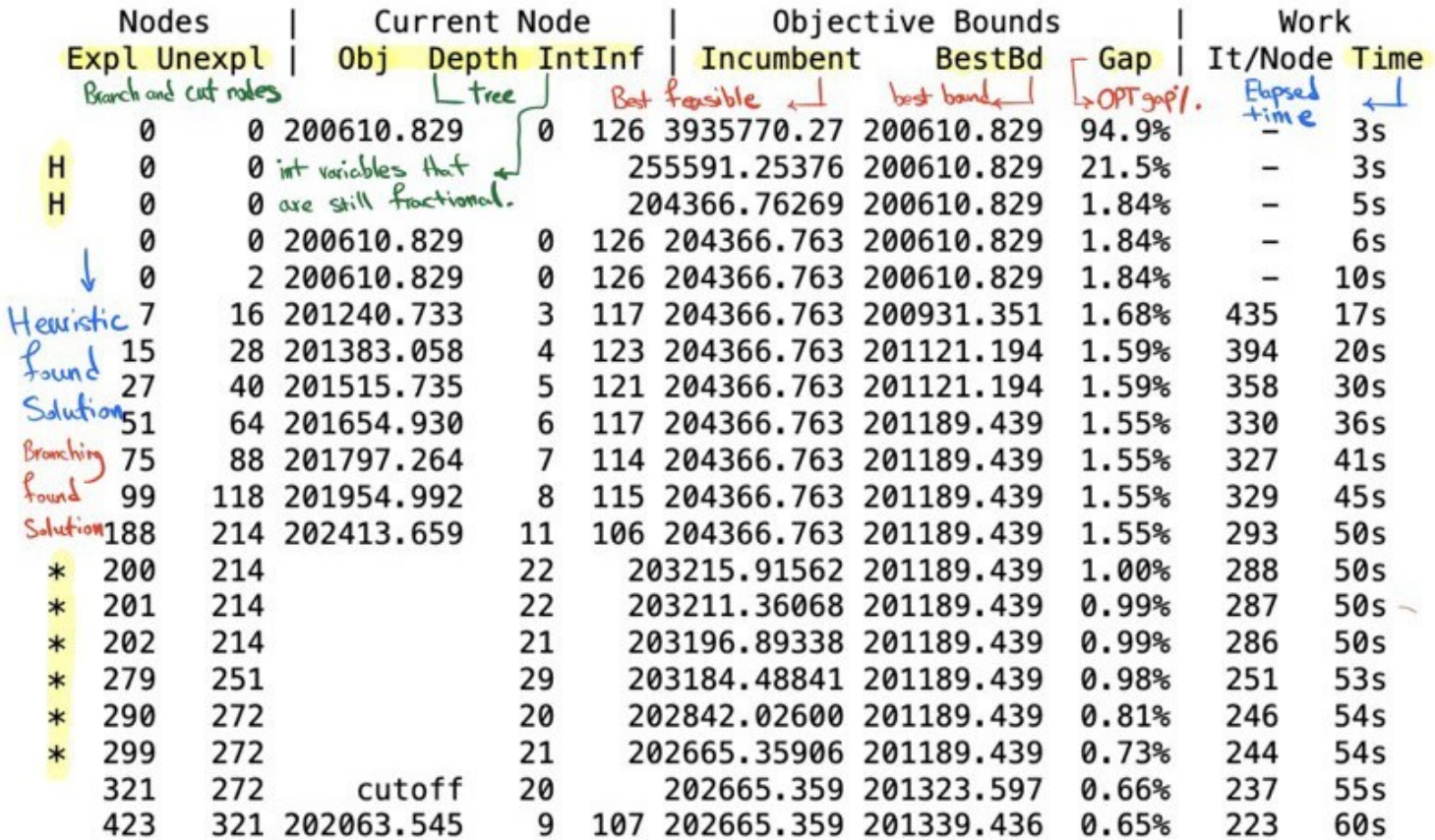
in this case, everything

Solving multiple Algs in parallel

Time that one Alg needed to wait so the other can finish running

Total time and iterations spent on Solving the linear relaxation of the problem.

Information in Gurobi Output



Information in Gurobi Output

```
Explored 3438 nodes (619701 simplex iterations) in 163.95 seconds (1021.80 work units)
Thread count was 12 (of 12 available processors)

Solution count 10: 202493 202544 202657 ... 204367

Optimal solution found (tolerance 1.00e-04)
Best objective 2.024932793509e+05, best bound 2.024932793509e+05, gap 0.0000%
```

Gurobi model as a function

- You can define a Gurobi model as a function that gets the parameters as input, and returns optimization results.
- Define the facility location problem as a function.
- If the model solved to optimality the function has to return the optimal objective value, the optimal opened facilities, and the optimal assignment of customers.
- Else, return dummy values for each (ex: -10^{-6})

Performance Tuning and Callbacks

- Gurobi allows for performance tuning by adjusting parameters.
- Callbacks can monitor solver progress (and many other things you might want to use later.)
- Example: setting the ‘MIPGap‘ parameter to control the precision of solutions.

Callback Example

- Define a callback function as follows:

```
def callback(model, where):
if where == GRB.Callback.MIPSOL:
    obj = model.cbGet(GRB.Callback.MIPSOL_OBJ)
    runtime = model.cbGet(GRB.Callback.RUNTIME)
    print(f"Time: {runtime:.1f}s, Obj: {obj:.2f}")
```

- Bring your code for the large-scale facility location problem.
- Instead of `model.optimize()`, use `model.optimize(callback)`

Performance Tuning Example

- You can set an allowable gap between the best integer solution found and the best bound on the optimal solution:

```
model.Params.MIPGap = 0.15
```

- You can set a time limit for a Gurobi model:

```
model.Params.TimeLimit = 180
```

- Increase the number of facilities to 300.
- Set these two parameters for the facility location problem with BigM, before `model.optimize()`.
- Check the `Status` of the model by conditioning on `GRB.TIME_LIMIT`.

Some Useful Parameters

- Gurobi uses presolve techniques to simplify the model before optimizing. You can disable this by setting `GRB.Param.Presolve` parameter to 0.

```
# Disable presolve (default is usually enabled)
model.setParam(GRB.Param.Presolve, 0)
```

- Try this on your code for facility location problem and compare the solution time.

Some Useful Parameters

- You can disable the Gurobi output by setting `GRB.Param.OutputFlag` to 0

```
# Suppress output by setting OutputFlag to 0
model.setParam(GRB.Param.OutputFlag, 0)
```

- Try this on your code for facility location problem.
- It is more useful when you have a Gurobi model inside loops.

Wrapping Up

- You can now solve any LP or MILP problem using Gurobi.
- For those of you doing research in Linear Optimization, you can do almost everything you want using Gurobi features (Ex, Benders Decomposition, Column Generation, Multi-Stage Stochastic Programming, etc.)
- For more advanced stuff, take a look at Gurobi documentation: `www.gurobi.com/documentation`